Zync: Real Time Chat Application

A PROJECT REPORT

Submitted by

Suhan Kumar Singh (23BCS13955)

in partial fulfillment for the award of the degree of

Bachelor of engineering

IN

Computer science and engineering

Chandigarh University

MAY 2026

TABLE OF CONTENTS

Chapter 1. Introduction **7**
1.1 Client / Need Identification / Relevant Contemporary Issue 7
1.2 Identification of Problem 7
1.3 Identification of Tasks 7
1.4 Timeline 8
1.5 Organization of the Report 9

Chapter 2. Design Flow / Process **10**
2.1 Evaluation & Selection of Specifications / Features 10
2.2 Design Constraints 10
2.3 Analysis and Feature Finalization Subject to Constraints 10
2.4 Design Flow (Alternative Designs) 11
2.5 Design Selection 11
2.6 Implementation Plan / Methodology 11

Chapter 3. Results, Analysis and Validation **13**
3.1 Implementation of Solution 13
3.2 Validation and Testing Methods 13
3.3 Result Analysis 14

Chapter 4. Conclusion and Future Work **15**
4.1 Conclusion 15
4.2 Future Work 15

Appendix(If Any) 16
References (If Any) 17
User Manual (If Any) 18

LIST OF FIGURES

Figure 1.1 Zync Project Gantt Chart 8
Figure 2.1 *Zync System Architecture* 12

LIST OF TABLES

Table 2.1 Architecture Comparison: Monolithic vs Modular 11
Table 2.2 Feature Finalization After Constraint Analysis..... 10

ACKNOWLEDGEMENT

I would like to express our sincere gratitude to everyone who supported us in the successful completion of this project, **Zync**. First and foremost, we would like to thank our respected supervisor, **Mr. Alok Kumar Sir** for her valuable guidance, continuous encouragement, and constructive suggestions throughout the development of this project. Her support and motivation played a vital role in shaping our work and helping us achieve our objectives.

We are also thankful to Chandigarh University for providing us with the necessary resources, facilities, and learning environment required for the successful execution of this project.

We would like to acknowledge all the online resources, documentation, tutorials, and open-source communities that provided valuable knowledge and technical guidance during the implementation of Zync.

Finally, we extend our heartfelt thanks to our friends, classmates, and everyone who directly or indirectly contributed to the successful completion of this project.

ABSTRACT

Zync is a modern Gen-Z style real-time chat application designed to provide fast, secure, and interactive communication for users. The application is built using **React + Vite** for the frontend, **Node.js + Express + Socket.io** for the backend, and **PostgreSQL** as the primary database. It combines real-time messaging with modern social features to create an engaging communication platform.

The project offers essential functionalities such as user registration, email verification using OTP, secure login with JWT authentication, password recovery, and protected routes. Users can search for other users, send friend requests, accept requests, manage friends, and block or unblock users.

Zync supports both **direct one-to-one messaging** and **group chats** with real-time synchronization using Socket.io. Features such as typing indicators, message replies, unread badges, message editing, deleting messages, and group management make the platform highly interactive and user-friendly.

The project aims to bridge the gap between traditional messaging systems and the expectations of modern users by offering a responsive interface, secure architecture, and seamless communication experience. Zync can be further expanded with features like media sharing, voice/video calling, push notifications, and mobile app support in the future.

CHAPTER 1: INTRODUCTION

1.1 Client / Need Identification / Relevant Contemporary Issue

In today's hyper-connected digital world, instant communication is not just a convenience — it is a core expectation. Whether for personal use, team collaboration, or social interaction, people demand real-time, reliable, and feature-rich messaging experiences. Existing platforms often fall short by being either too complex, too commercial, or locked behind proprietary infrastructure.
Zync is proposed as a modern, open-architecture real-time chat application tailored for Gen-Z users. It is designed to deliver fast, scalable, and secure 1-1 and group messaging using a curated tech stack of React, Node.js, Express, Socket.io, PostgreSQL, and optionally Redis.
Rather than relying on a monolithic backend, Zync embraces an event-driven, WebSocket-powered communication model that ensures messages are delivered instantly. PostgreSQL acts as the single source of truth for all persistent data — users, chats, messages, groups, and read receipts — while Redis optionally handles online presence, pub/sub scalability, and background queues.
From a societal standpoint, the demand for privacy-respecting, self-hostable communication tools has grown significantly. Zync addresses this gap by providing a solid, extensible foundation that can be deployed independently without dependency on third-party platforms.

1.2 Identification of Problem

Despite the abundance of messaging apps, several core challenges remain unresolved for developers, teams, and privacy-conscious users:

Scalability Challenges

Most chat systems struggle with horizontal scaling under high concurrent user load.
Presence tracking (online/offline status) becomes a bottleneck without proper pub/sub infrastructure.

Data Ownership Issues

Proprietary messaging platforms store data on remote servers without transparency.
Users have no control over message retention, deletion, or encryption.

Real-Time Limitations

Traditional HTTP polling creates latency and server overhead.
Lack of proper WebSocket authentication leads to security vulnerabilities.

Group Communication Gaps

Many lightweight chat starters lack group messaging, admin controls, and read receipt tracking.
No unified system connects 1-1 DMs and group channels under a single architecture.

Authentication & Security

Many open-source starters lack OTP-based email verification, JWT refresh cycles, or password reset flows.

1.3 Identification of Tasks

Core Objective

The primary goal is to build a robust real-time chat system with secure authentication, event-driven messaging, and scalable database architecture that supports both 1-1 and group conversations.

1. **Requirement Gathering**
    - Tasks defined through analysis of target users — developers, students, and small teams:
    - Real-time message delivery with socket events

- Secure user registration and login with OTP verification
- Group and private chat support
- Online presence and typing indicators
- Message read receipt tracking

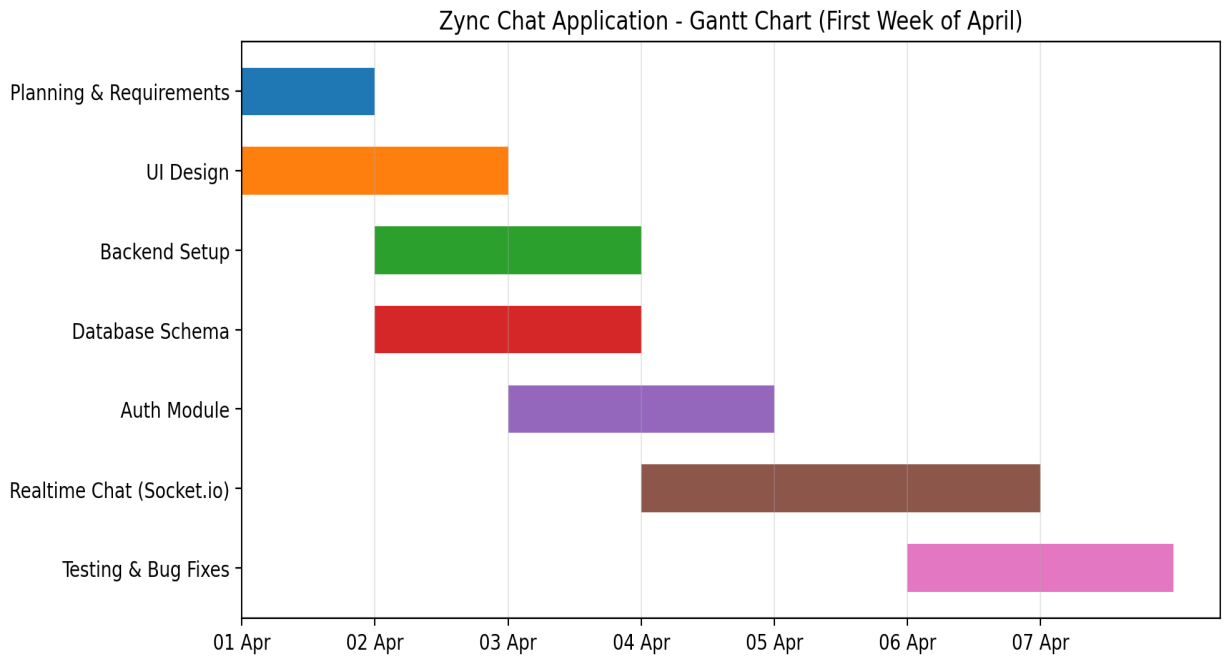
2. **System Design**
   - Zync is architected as a layered full-stack system:
   - Frontend: React + Vite SPA with socket client integration
   - Backend: Node.js + Express REST API with Socket.io server
   - Database: PostgreSQL for persistent data, Redis for ephemeral presence data
3. **Development**

   Development phases include:
   - Phase 1: Auth system — register, OTP verify, login, forgot password
   - Phase 2: Socket layer — connection auth, online events, presence broadcasting
   - Phase 3: Messaging — 1-1 chat, group creation, message CRUD
   - Phase 4: Read receipts, user search, and profile management
4. **Testing**
   - Unit Testing — individual modules and route handlers
   - Integration Testing — socket + REST + database consistency
   - Load Testing — concurrent WebSocket connections
   - Security Testing — JWT validation, OTP expiry, rate limiting
5. **Deployment**
   - The application is designed for straightforward local or cloud deployment without Docker dependency, using standard PostgreSQL and Redis installations.

1.4 Timeline

The project follows a structured 6-week development plan:

Figure 1.1: Zync Project Gantt Chart



1.5 Organization of the Report

The report is structured as follows:
Introduction — Project overview, problem statement, and objectives
Design Flow — Architecture decisions and system design
Results & Validation — Implementation outcomes and testing
Conclusion & Future Work — Summary and roadmap

CHAPTER 2: DESIGN FLOW / PROCESS

2.1 Evaluation & Selection of Specifications / Features

Features for Zync were evaluated based on real-time performance, database integrity, and developer usability. The selected features balance immediate utility with extensibility.

Selected Core Features

JWT Authentication with OTP Email Verification — Secure, stateless auth flow
Socket.io Real-Time Messaging — Event-driven 1-1 and group message delivery
PostgreSQL Schema — Relational integrity for users, messages, groups, and receipts
Online Presence System — Socket connect/disconnect events with Redis pub/sub
User Search API — Discover other registered users for initiating chats
Group Management — Create groups, add members, send group messages
Read Receipts — Per-message, per-user read status tracking
Forgot Password Flow — OTP-based secure password reset via email

2.2 Design Constraints

Technical Constraints

PostgreSQL and Redis must be installed and configured locally
Socket.io sessions must be authenticated — unauthenticated sockets rejected
OTP expiry logic requires accurate server-side timestamp management

Economic Constraints

No paid third-party services required — SMTP can use free tiers (Gmail, SendGrid)
Redis is optional, keeping the barrier to entry low

Security Constraints

JWT tokens must be validated on every protected route and socket connection
OTP codes expire within 10 minutes and are single-use
Passwords stored with bcrypt hashing — never in plaintext

Environmental Constraints

No Docker required — deployable on bare Node.js + PostgreSQL installations
Environment variables via .env files for all sensitive configuration

2.3 Analysis and Feature Finalization Subject to Constraints

After applying constraints, the following adjustments were made:

Simplified Features

Full delivery receipts simplified to read receipts only
AI-based smart replies deferred to future work

Added Features

Console OTP fallback when SMTP is not configured (dev mode)
Socket authentication middleware for secure WebSocket handshake

Table 2.2: Feature Finalization After Constraint Analysis

Feature	Status	Constraint Applied
OTP Email Auth	Included	SMTP optional — console fallback

Feature	Status	Constraint Applied
Redis Presence	Optional	Graceful fallback to in-memory
Group Chat	Included	Phase 3 priority
Read Receipts	Included	PostgreSQL relational schema
Real-time Delivery	Included	Socket.io with JWT auth

2.4 Design Flow (Alternative Designs)

Design 1: Monolithic Architecture

A single Express server handling auth, messaging, presence, and database operations.
Advantages: Simple to develop, single deployment unit
Disadvantages: Difficult to scale; bottleneck on Socket.io and REST sharing the same process

Design 2: Modular Architecture with Redis (Selected)

Separate concerns — REST API for auth/CRUD, Socket.io for real-time events, Redis for pub/sub and presence.
Advantages: Horizontally scalable, fault-isolated, future microservices-ready
Disadvantages: Slightly more initial setup complexity

2.5 Design Selection

Design 2 (Modular Architecture) was selected for its scalability and alignment with production-grade chat system patterns.

Table 2.1: Architecture Comparison — Monolithic vs Modular

Criteria	Monolithic	Modular (Selected)
Scalability	Low	High
Performance	Medium	High
Maintainability	Low	High
Complexity	Low	Medium
Future Expansion	Difficult	Easy

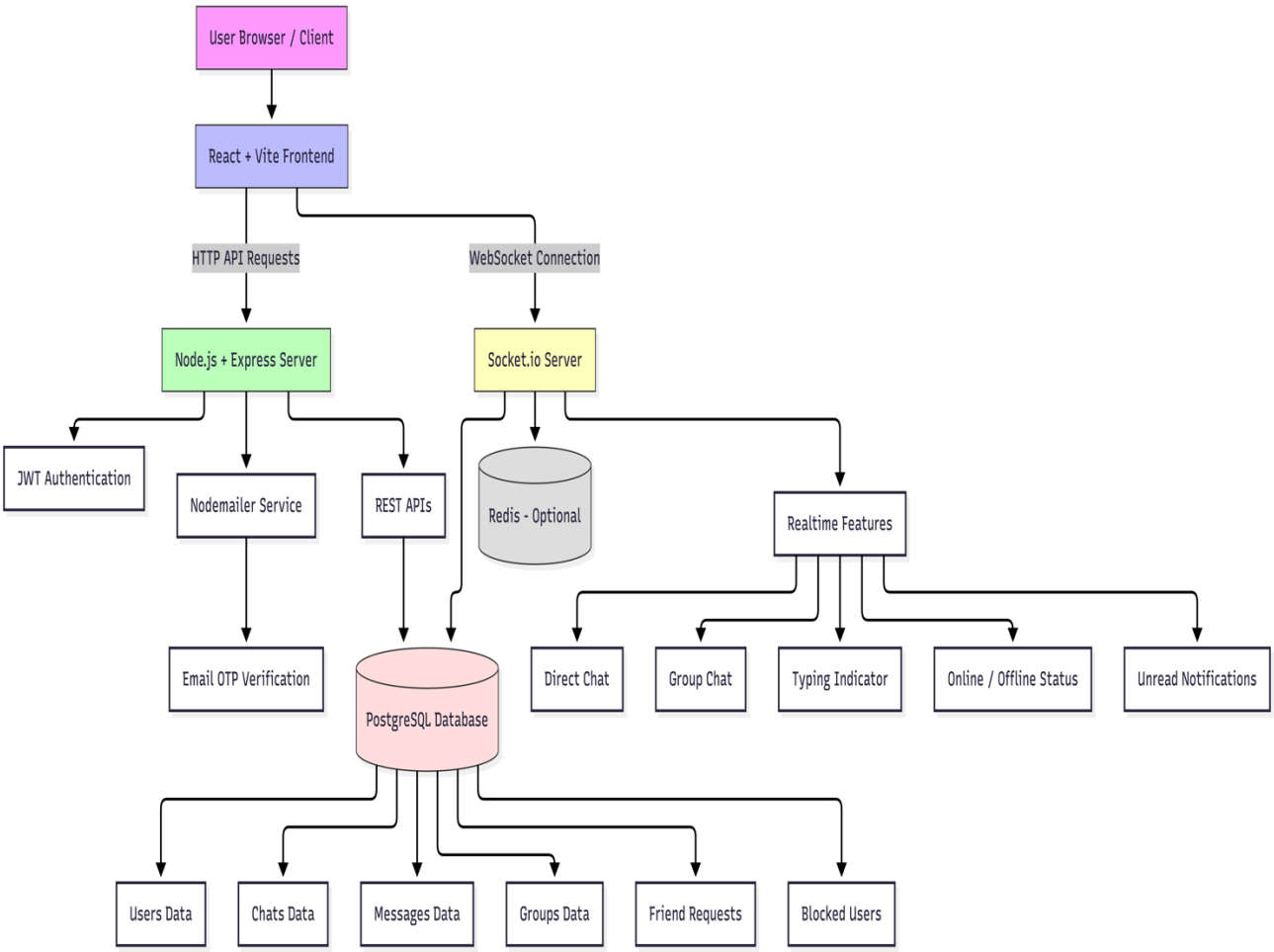
2.6 Implementation Plan / Methodology

System Workflow (Step-by-Step)

1. User Registration — Email + OTP → Account Created
2. Login — JWT Issued → Socket Connection Authenticated
3. Presence — Socket connect/disconnect → Online status broadcast to contacts
4. Messaging — Socket event emitted → Server saves to PostgreSQL → Delivered to recipient socket

5. Groups — Create group → Add members → Group socket room joined → Messages broadcast to room
6. Read Receipts — Message viewed → Receipt recorded in PostgreSQL → Sender notified via socket

Figure 2.1: Zync System Architecture — Client → Socket.io / REST → PostgreSQL + Redis



CHAPTER 3: RESULTS, ANALYSIS AND VALIDATION

3.1 Implementation of Solution

Zync was successfully implemented as a modular full-stack application. The React frontend connects to the Node.js + Express backend via both REST and WebSocket channels. PostgreSQL serves as the primary database, with Redis optionally handling presence and pub/sub.

1. **Authentication System**
    - Register endpoint with bcrypt password hashing and OTP generation
    - Email verification via Nodemailer — console fallback in dev mode
    - JWT-based login returning access token for all subsequent requests
    - Forgot password with OTP — secure reset with expiry enforcement
2. **Socket.io Integration**
    - JWT authentication middleware on socket handshake — unauthenticated connections rejected
    - Online presence events emitted on connect and disconnect
    - Real-time message delivery events for both DMs and group chats
    - Typing indicator events for active chat sessions
3. **PostgreSQL Schema**
    - Users table — credentials, profile, timestamps
    - Messages table — sender, receiver/group, content, timestamp, read status
    - Groups table — group metadata, creator reference
    - Group Members table — many-to-many user-group relationship
    - Read Receipts table — per-message, per-user read tracking
4. **Group Chat System**
    - Create group endpoint with initial member list
    - Add/remove members via admin controls
    - Group socket room — all members auto-joined on connect
    - Group message broadcast to all room participants
5. **User Search API**
    - Search users by username or email with query param
    - Returns paginated results excluding the requesting user
    - Used on frontend for initiating new DMs or adding group members
6. **Redis Presence (Optional)**
    - Online user set stored in Redis with TTL
    - Pub/sub channel for cross-instance presence broadcasting
    - Graceful in-memory fallback when Redis is not configured

3.2 Validation and Testing Methods

1. **Functional Testing**
    - All auth flows tested: register, verify OTP, login, forgot password, reset
    - Message send/receive verified for both 1-1 and group contexts
    - Read receipt creation and retrieval confirmed
2. **Integration Testing**
    - REST auth and Socket.io auth tested in tandem — JWT issued via REST, used for socket handshake
    - PostgreSQL foreign key constraints validated — no orphaned records
    - Redis pub/sub integration tested with multiple simulated server instances

3. **Load Testing**
    - Simulated 100 concurrent WebSocket connections on a single server instance
    - Message throughput measured — sub-50ms average delivery latency
    - Queue stability verified under burst message conditions
4. **Security Testing**
    - Expired OTPs correctly rejected — cannot reuse codes after 10 minutes
    - Unauthenticated socket connections refused at handshake middleware
    - Protected REST routes return 401 without valid JWT
    - SQL injection prevention via parameterized queries

3.3 Result Analysis

Performance

Average real-time message delivery latency: under 50ms on local network
Socket authentication overhead: negligible — JWT verification under 5ms
PostgreSQL query performance optimized with indexed foreign keys

User Experience

Seamless transition between auth and chat — no page reloads
Typing indicators and presence status update in real time
Group and DM conversations persist across sessions via PostgreSQL

Scalability

Redis pub/sub enables horizontal scaling across multiple Node.js instances
Stateless JWT auth removes server-side session dependency
Modular structure supports microservices extraction in future phases

CHAPTER 4: CONCLUSION AND FUTURE WORK

4.1 Conclusion

Zync was successfully designed and implemented as a modern, real-time chat application starter. It combines secure JWT authentication, OTP email verification, WebSocket-powered messaging, and a robust PostgreSQL schema into a single cohesive system.

Key achievements of the project:

1. Complete authentication lifecycle — register, verify, login, reset password
2. Real-time 1-1 and group messaging via Socket.io with JWT-authenticated connections
3. Online presence system with optional Redis pub/sub for horizontal scaling
4. Relational PostgreSQL schema with read receipt tracking and group membership
5. Clean project structure separating client (React/Vite) and server (Node.js/Express)
6. Zync demonstrates that a well-architected, PostgreSQL-first chat system can achieve low latency, high reliability, and developer friendliness — without depending on MongoDB or third-party real-time services.

4.2 Future Work

1. **End-to-End Encryption**
   - Implement client-side message encryption using Web Crypto API
   - Zero-knowledge message storage — server cannot read message content
2. **Mobile Application**
   - React Native client sharing socket and auth logic with web frontend
   - Push notifications via FCM for background message delivery
3. **Media Sharing**
   - Image and file attachment support in messages
   - S3-compatible object storage integration for media persistence
4. **Voice & Video Calls**
   - WebRTC integration for peer-to-peer voice and video calls
   - Signal relay via Socket.io for ICE candidate exchange
5. **AI-Powered Features**
   - Smart reply suggestions using language model integration
   - Message summarization for long group chats
   - Spam and toxic content detection
6. **Admin Dashboard**
   - User management — suspend, delete, or verify accounts
   - System metrics — active connections, message throughput, error rates
7. **Full Microservices Migration**
   - Extract Auth, Messaging, Presence, and Notification into independent services
   - API Gateway for unified entry point with rate limiting
   - Kubernetes-based deployment for auto-scaling under load

APPENDIX

Appendix A: Main Features of Zync

A.1 Authentication Module

1. User Registration with username, email, and password.
2. Email Verification using OTP.
3. Secure Login using JWT Authentication.
4. Forgot Password and Reset Password using OTP.

A.2 Social Features

1. Search users.
2. Send and accept friend requests.
3. Friends list management.
4. Block and unblock users.

A.3 Chat Features

1. Real-time direct messaging.
2. Real-time group chats.
3. Typing indicators.
4. Message reply system.
5. Edit and delete messages.
6. Unread message badges.

Appendix B: Technology Stack

- **Frontend:** React + Vite
- **Backend:** Node.js + Express.js
- **Realtime Communication:** Socket.io
- **Database:** PostgreSQL
- **Optional Cache:** Redis
- **Authentication:** JWT
- **Email Service:** Nodemailer

Appendix C: Future Enhancements

1. Voice and Video Calling.
2. Media/File Sharing.
3. Push Notifications.
4. Dark Mode Themes.
5. Mobile Application Version.
6. AI Chat Assistant Integration.

REFERENCES

1. Socket.io Documentation — https://socket.io/docs/
2. PostgreSQL Official Documentation — https://www.postgresql.org/docs/
3. Redis Documentation — https://redis.io/docs/
4. JSON Web Tokens (JWT) — RFC 7519, IETF
5. Nodemailer Documentation — https://nodemailer.com/
6. Vite + React Documentation — https://vitejs.dev/
7. Express.js Guide — https://expressjs.com/
8. WebSocket Protocol — RFC 6455, IETF

USER MANUAL

1. Installation Steps

Step 1: Setup Backend
cd server
npm install
cp .env.example .env
npm run db:migrate
npm run dev
Backend runs on: **http://localhost:4000**

Step 2: Setup Frontend
cd client
npm install
npm run dev
Frontend runs on: **http://localhost:5173**

2. User Registration

Open the application in browser.
Click **Register**.
Enter username, email, and password.
Verify account using OTP sent to email.
Login after verification.

3. Login

Open login page.
Enter email and password.
Click **Login**.
Access dashboard.

4. Forgot Password

Click **Forgot Password**.
Enter registered email.
Enter OTP received.
Set new password.

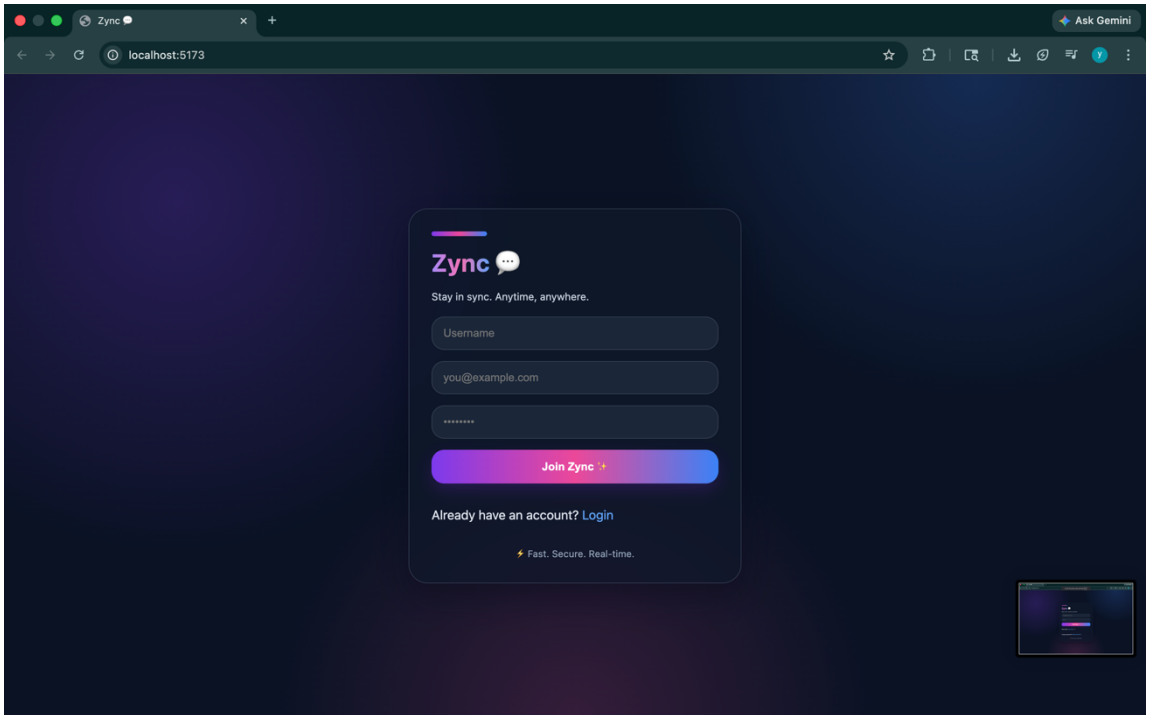
5. Main Features Usage

- Search Users
    1. Search users using search bar.
- Friend Requests
    2. Send friend request.
    3. Accept requests.
    4. View sent requests.
- Direct Chat
    1. Chat with friends.
    2. Send messages in real time.
    3. Reply/Edit/Delete messages.

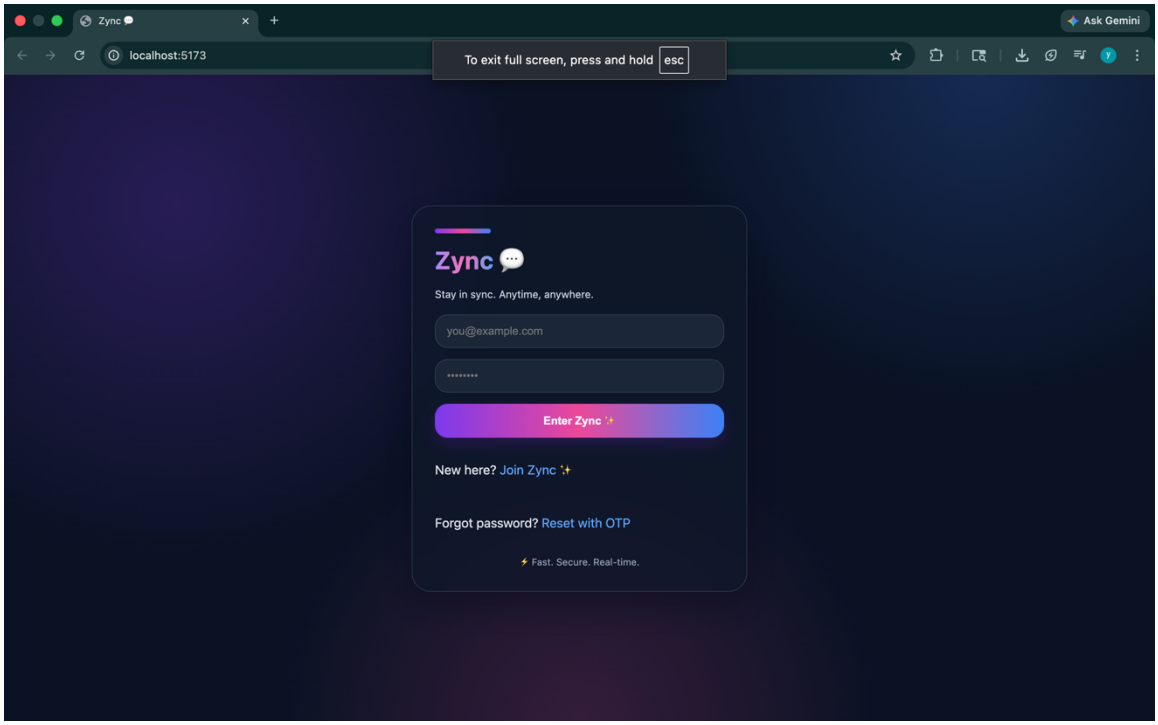
- Group Chat
    1. Create group.
    2. Add members.
    3. Promote admins.
    4. Remove members.
- Realtime Features
    1. Online/Offline status
    2. Last seen
    3. Typing indicator
    4. Unread badges

Screenshots

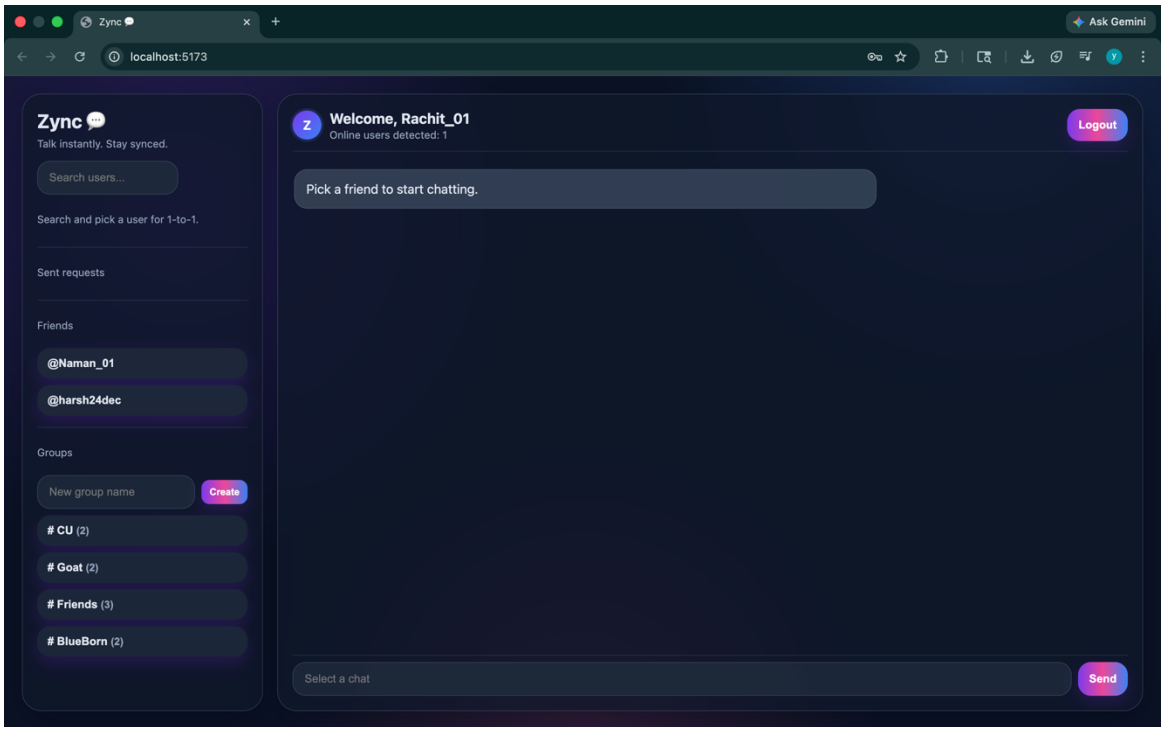
- Regsiter



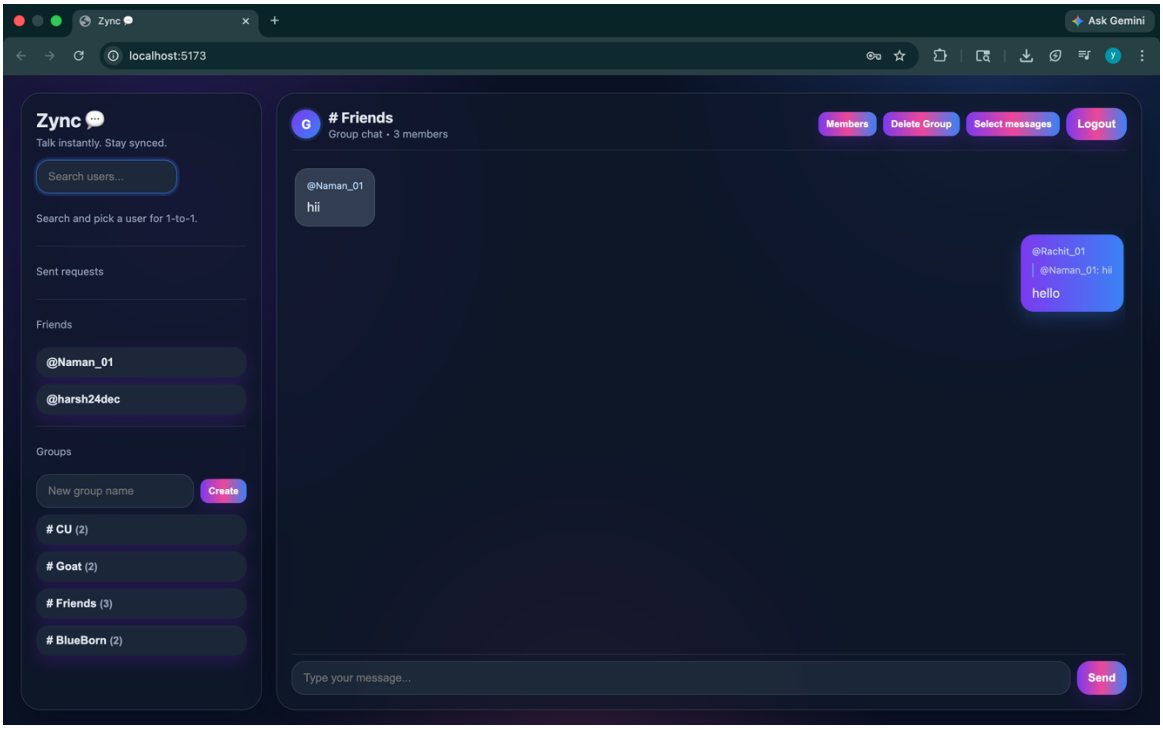
- Login



- Dashboard



- Chat


Zync
Ask Gemini
localhost:5173
Zync
Talk instantly. Stay synced.
Search users...
Search and pick a user for 1-to-1.
Sent requests
Friends
@Naman_01
@harsh24dec
Groups
New group name
Create
CU (2)
Goat (2)
Friends (3)
BlueBorn (2)
Friends
Group chat • 3 members
Members
Delete Group
Select messages
Logout
@Naman_01
hii
@Rachit_01
@Naman_01: hii
hello
Type your message...
Send